Information Technology Course
Module AIS
Prof. Dr. Peter Nauth

Enhancing ROSWITHA's visual perception through deep learning which is based on 10-Class segmentation model

Rajan Singh
Frankfurt University of Applied Sciences
1567294
rajan.singh@stud.fra-uas.de

Pradeep Tiwari
Frankfurt University of Applied Sciences
1567166
pradeep.tiwari@stud.fra-uas.de

Abstract*—Visual perception is a significant difficulty in assistive robotics, especially when robots are required to function effectively in intricate indoor settings alongside people. This paper outlines our efforts to develop a resilient RGB-D (depth) semantic segmentation system. Frankfurt University of Applied Sciences (Frankfurt UAS) has developed its own autonomous robot platform, the assistive robot "ROSWITHA" (RObot System WITH Autonomy). We improved the robot's ability to see by adding three more item classes, bringing the total from 7 to 10. These new classes include laptops, backpacks, and microwave ovens, which are useful for household tasks. We use a unique U-Net-based encoder-decoder architecture that processes six input channels simultaneously to integrate RGB colour information with depth data (XYZ coordinate). Our trials demonstrated a consistent tension: forceful efforts to improve performance in rare courses invariably led to catastrophic forgetting, resulting in a decline of 15-40 percentage points in classes that had previously performed well. This discovery prompted us to create a cautious training technique that emphasizes stability rather than aggressive optimization. This article offers practical insights for robotics practitioners and scientific evidence on the inherent constraints of deep learning under conditions of severe data scarcity. Our findings indicate that for classes that are severely underrepresented, making up less than 0.01% of training pixels, even advanced algorithmic methods finds it challenging to fully make up for the lack of data.

***Index Terms*—ROSWITHA, Semantic segmentation, RGB-D perception, assistive robotics, class imbalance, catastrophic forgetting, deep learning, ROS2, U-Net architecture, conservative training, focal loss**

I. INTRODUCTION

A. Motivation and Context

The worldwide trend of population ageing has made it imperative to develop assistive technologies that help people stay independent in their daily lives. Assistive robots are a promising solution, but they only work well if they can visually envision. Frankfurt University of Applied Sciences (Frankfurt UAS) has developed its own autonomous robot platform, the assistance robot "ROSWITHA" (RObot System WITH Autonomy), Fig. 1, made to help people with things like finding objects, keeping an eye on their surroundings, and getting around their homes. For this kind of robot to be useful, it needs to be able to find and identify everyday objects, even in a messy space, under different lighting conditions, and in rooms with more obstacles. Earlier versions of the ROSWITHA perception system could distinguish among 7 fundamental object types. This inspired our research to broaden the perception capabilities to 10 categories while still working in real time.

B. Research Outcome

Adding 3 more classes to semantic segmentation may seem like a simple job, yet we ran into a few basic problems that affected our whole research method: real-world data shows a severe class imbalance. When we looked into our training dataset, we found statistical patterns that were very different from what seems normal. 94.15% of the labelled data consists of background pixels, while Backpacks, a rare class, appear in only 0.0035% of pixels. This yields an imbalance ratio of 26,900:1, which is much worse than the usual 10:1 to 100:1 range discussed in semantic segmentation literature [1] [2]. Regular Training practices just didn't work in these situations, with Models quickly learning to guess the "background" for every pixel. At first, we tried to fix the class imbalance using harsh methods: very high-class weights (5000×, compounding rare-class losses) and oversampling (rare events 20× more often). These methods did make the model learn unusual classes, but it will cost a lot. Classes that had been 70–80% accurate at first decreased to 15% or 40%, basically forgetting everything they had studied about common things. This phenomenon, known as catastrophic forgetting [3], [4], became the main problem we had to fix. Not enough training data for rare classes. The total number of pixels available for unusual classes is insufficient to train the model. For example, backpacks make up only 0.0035% of the pixels in our entire training set. There are about 450,000 backpack pixels, which is

equivalent to 27 full pictures at our 128×128 resolution. This amount is not enough for learning strong visual elements, especially when you think about the variance in position, lighting and occlusion patterns within a class. Standard data augmentation methods only provided a slight improvement.

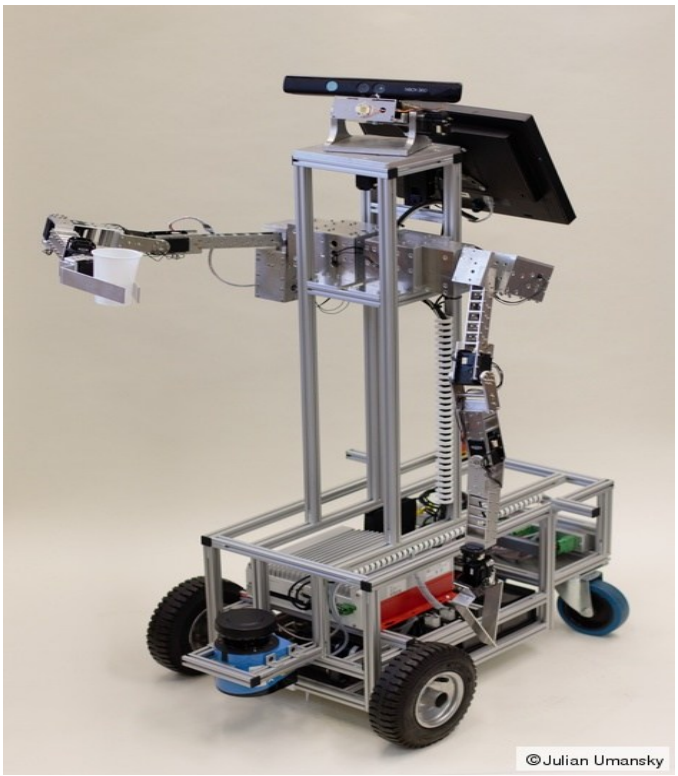

Fig. 1: ROSWITHA still [41]

C. Contributions

This project adds a lot to the realm of robotics perception: At first, we begin with a thorough examination of training, methodologies for semantic segmentation in the presence of severe class imbalance, recording both effective methods and challenges and fundamental restrictions. Our system of fourteen training sessions systematically looks into the area of probable solutions and makes it obvious to show the pros and cons of each option.

Secondly, we present a cautious training approach with clear safety checks that stop catastrophic forgetting while making small progress in rare classes. This strategy found the finest balance we could find between goals that are at odds with each other. We have provided a full implementation, integrated with ROS2, demonstrating that it can be put into practice on assistive robots. Lastly, we provide honest records of our failures, modes, which we think are useful for researchers looking into comparable issues. Not every class may be learned well from our dataset, and we clearly find out which ones and why.

II. RELATED WORK

A. Semantic Segmentation Architectures

Semantic segmentation has changed quickly over time since Fully Convolutional Networks were first used (FCNs) [5]. Initial methodologies, such as SegNet [6], demonstrated that encoder-decoder designs were capable of achieving pixel-wise classification but struggled to preserve fine details. The U-Net architecture [7], originally developed for biomedical image segmentation, introduced skip connections that connect the encoder and decoder layers, thereby greatly improving detail preservation. This architectural style has served as a foundation for robotics and medical imaging.

More contemporary architectures have added upon these basic ideas. DenseNet [8] introduced dense connections, in which each layer receives inputs from all layers preceding it, encouraging the flow of gradients and the reuse of features. FC-DenseNet [9] changed this notion for semantic segmentation by integrating it into a complete convolutional framework. FC-HarDNet [10] goes even further to improve these principles by adding a tightly connected network that uses less memory without hampering the accuracy of the model.

For RGB-D segmentation in particular, numerous architectures have been proposed. There are now architectures that expressly deal with multi-modal fusion MMAF-Net [11]. It presents a Multi-Modal Adaptive Fusion method that learns to give different weights to RGB and depth features based on the context of the scene. This was especially true and works well in indoor scenario where in-depth information is useful which can settle unclear issues that colour alone can't fix.

B. Attention Mechanisms in Deep Learning

Attention mechanisms are becoming increasingly crucial in the field of computer vision. The Squeeze-and-Excitation Network [12] presented channel-wise attention that recalibrates itself in a flexible way, feature maps by explicitly modelling interdependencies between channels. EfficientNet [13] showed that combining attention mechanisms with careful architectural scaling can lead to better trade-offs between accuracy and performance than earlier methods.

In the case of segmentation, attention methods assist the model in focusing on the features that matter and ignoring the ones that don't. This is especially helpful when there is an imbalance in the classes, because the model can learn to pay greater attention to rare but essential things.

C. Catastrophic Forgetting and Continual Learning

Researchers have investigated catastrophic forgetting, which happens when neural networks forget something they learnt while they are trained on new tasks [3]. Elastic Weight

Consolidation [4] solves this by limiting changes to essential parameters that were found in earlier tasks. Progressive Neural Networks [14] keep separate capacity for each task, which keeps them from interfering with each other but makes the model bigger.

In our case, the "tasks" are not done one after the other, but at the same time. We want the model to do well on all of them at the same time. Our topic is a little different from most continual learning problems, but the basic ideas behind how conflicting goals interfere with each other are still useful.

III. METHODOLOGY

A. Overall Architecture Design

We built our semantic segmentation model by carefully reviewing successful architectures from recent literature and selecting components that fit our needs. Fig. 2, illustrates the hierarchical framework of our FRA-UAS-SS (Frankfurt University of Applied Sciences Semantic Segmentation) model and its correlation with basic structures.

The architecture is made up of three main parts that function together: a backbone network that extracts features, a multi-modal fusion module that combines RGB and depth information, and an attention mechanism that adapts to improve features. We chose each part depending on how well it worked in practice and whether it fit inside our computational limits. Fig.1, The backbone comes from FC- HarDNet, the RGB-D fusion comes from MMAF-Net, and the attention mechanisms come from EfficientNet.

B. Backbone Network: From CNNs to FC-HarDNet

The backbone network used for getting useful information from the images that are input. Our approach is based on a long line of convolutional designs, each one fixing problems that the ones before it had.

Foundational CNN Architecture: Convolutional Neural Networks [15] introduced the key ideas that we still use today: local connection, weight sharing, and hierarchical feature extraction. These traits make CNNs great for interpreting images.

Encoder-Decoder Framework: For dense prediction tasks such as semantic segmentation, it is necessary to preserve spatial resolution while extracting semantic characteristics. The encoder-decoder design [6] solves this problem by employing a contracting path (the encoder) to gather context and an expanding path (the decoder) to allow for exact localization. The encoder gradually decreases the spatial dimensions and increases the feature depth, while the decoder does the opposite to make predictions for each pixel

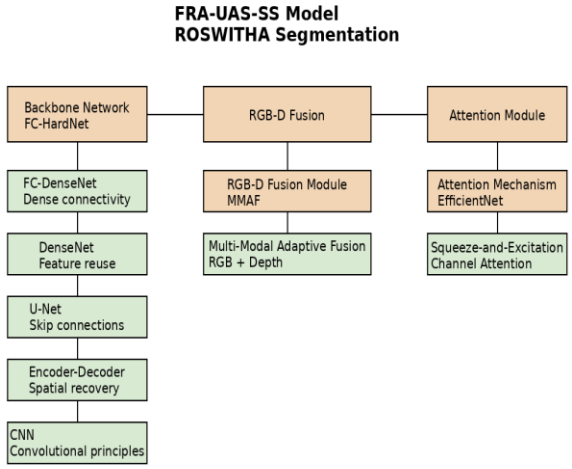

Fig. 2: Hierarchical architecture of the FRA-UAS-SS model, showing the relationships between our model and foundational architectures [37]

U-Net with Skip Connections: A significant drawback of fundamental encoder-decoder systems is the degradation of intricate spatial features during the encoding phase. U-Net [7] addressed the issue by adding skip connections that link the encoder and decoder layers. These connections let the decoder obtain high-resolution information from the early encoder layers, making it much easier to find boundaries. We employ skip connections in four different ways in our approach. 128×128, 64×64, 32×32, and 16×16 are different resolution settings.

DenseNet Connectivity Pattern: Skip connections, which only connect certain pairings of layers. DenseNet [8] created a more aggressive style of connectedness in which each layer gets feature maps from all of the layers that came before a dense block. This extensive connectedness has a few different uses: It makes gradient propagation stronger during training and encourages feature reuse, and cuts down on the number of parameters needed. Each layer can access features at all levels of abstraction instead of instead of just the layer just before it.

FC-DenseNet Adaptation: FC-DenseNet [9] changed the DenseNet and organizes the idea of semantic segmentation by dense chunks in a framework that encodes and decodes. The encoder has thick blocks with downsampling in between them, The decoder has dense blocks that are upsample. Transition layers between blocks handle the changes in dimensionality. This architecture got great results on a number of benchmarks, but memory usage became an issue for networks that are deeper.

FC-HarDNet Optimization: Our last choice for a back- bone is FC-HarDNet [10] fixes the memory problems of FC-

DenseNet while keeping its benefits. HarDNet brings in ”Harmonic” connection means that each layer links to a logarithmically selection of prior layers that are spaced out in an arithmetic way instead of all of them. This lowers the amount of memory bandwidth used by about 40% less than DenseNet while still getting similar precision. For the deployment of our robot, we must fit the model on embedded GPUs, which will make it more efficient, which turned out to be quite important.

In our implementation, the encoder has four HarDNet blocks with channels [64, 128, 256, 512] that use max pooling to make things smaller. The decoder copies this structure. uses transposed convolutions for upsampling. Every dense block, there are 4 to 6 layers in a block, and the growth rate is 32 channels.

C. Multi-Modal Fusion: MMAF-Net Approach

In indoor settings, there are situations where RGB information by itself is unclear. A white chair might not stand out; it might look like a white table. But the depth shows the difference in height. On the other hand, depth could show two things. At the same distance, a red bag stands out from a purple bottle because of its colour. In RGB-D segmentation, we combined the two modes. Naive Concatenation: The simplest way to fuse RGB and depth channels is to combine them into a 6-channel input (RGB + XYZ). This lets the network go reach both modalities. Thus, it depends on the network to learn the modality without being told. Our initial trials demonstrated that this method did not work as well as it could have, especially in classes where one modality was particularly revealing, such as using depth to find “free space”.

MMAF-Net Architecture: We took the Multi-Modal Adaptive Fusion method from MMAF-Net [11], which processes RGB and depth streams individually before fusing. The RGB stream goes through three convolutional layers (64, 128, and 256 channels), and the depth stream goes through a parallel path. A fusion module combines the streams that use learning attention weights to decide how important one modality is compared to the others. The fusion module calculates attention weights via a tiny network that works with RGB and depth, which are put together. For features $F_{RGB} \in \mathbb{R}^{\mathbb{C}\times\mathbb{H}\times\mathbb{W}}$ and $F_{depth} \in \mathbb{R}^{\mathbb{C}\times\mathbb{H}\times\mathbb{W}}$ The fusion works as follows:

$$F_{concat} = [F_{RGB}; F_{depth}] \quad (1)$$

$$\alpha = \sigma\,(W2 \cdot \mathrm{ReLU}\,(W1 \cdot GAP(F_{concat}))) \quad (2)$$

$$F_{fused} = \alpha \odot F_{RGB} + (1-\alpha) \odot F_{depth} \quad (3)$$

where GAP stands for global average pooling, W1 and W2 are learned parameters, σ is sigmoid activation, and α ∈ [0, 1]C provides channel-wise attention. This lets the model for each feature channel, you can give RGB and depth varying weights. changing to what gives the greatest information for the current scene.

D. Attention Mechanisms: Squeeze-and-Excitation

Not all feature channels are present even after multi-modal fusion which is equally important for the final prognosis in the same way. Attention mechanisms help the model focus on the most important elements.

Channel Attention with SE Blocks: We add at several locations, there are Squeeze-and-Excitation (SE) blocks [12] in our architecture, based on EfficientNet design rules. An SE block recalibrates channel characteristics by clearly modelling how they relate to each other. Given feature maps $X \in \mathbb{R}^{\mathbb{C}\times\mathbb{H}\times\mathbb{W}}$, the SE block starts with” compressing” global spatial information into a channel description by pooling the average of all the data:

$$z_c = \frac{1}{H\times W}\sum_{i=1}^{H}\sum_{j=1}^{W} X_{c,i,j} \quad (4)$$

The “excitation” function then learns channel-wise important

through a two-layer network:

$$s = \sigma\big(W_2 \cdot ReLU(W_1 \cdot z)\big) \quad (5)$$

where $W_1 \in \mathbb{R}^{\frac{\mathbb{C}}{\mathbb{r}}\times\mathbb{C}}$ and $W_2 \in \mathbb{R}^{\mathbb{C}\times\frac{\mathbb{C}}{\mathbb{r}}}$ with reduction ratio r = 16. Finally, the features are changed:

$$\widetilde{X_c} = s_c \cdot X_c \quad (6)$$

We kept SE blocks after each dense block in both the encoder and the decoder. We saw that SE blocks were in use throughout the training and deeper layers learned to focus on traits that match uncommon object classes, which effectively make up for class imbalance to a certain extent.

E. Loss Function Design

Our loss function combines several goals to solve both the quality of segmentation and the imbalance in the class:

Focal Loss Component: The focal loss [1] talks about class imbalance by lowering the impact of simple examples:

$$L_{focal} = -\alpha_c(1-p_c)^{\gamma}\log(p_c) \quad (7)$$

where pc is the predicted probability for the right class, αc is the weight of the class, and γ = 2 is the parameter that focuses. We set the weights for class weights αc inversely related to class frequency, but limited to the highest settings to stop steep slopes.

Dice Loss Component: The Dice loss overlaps as well as it can be between the prognosis and the real world:

$$L_{dice} = 1 - \frac{2\sum_i p_i g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon} \quad (8)$$

where p_i and g_i are projected and actual prob ability, and $\epsilon = 1$ stops division by zero. We figure out the Dice loss per-class and average with class weights.

Combined Loss: Our total loss is the sum of both terms:

$$L_{total} = \lambda_{focal} L_{focal} + \lambda_{dice} L_{dice} \quad (9)$$

We employ λfocal = 0.7 and λdice = 0.3 based on testing performance.

F. Training Strategy Evolution

We went through fourteen training sessions to systematically remake our plan. Classes 1 through 8 had very great difficulty weights (5000× for rare classes) and fast learning rates (1e-3), leading to catastrophic forgetting. The middle sessions (9-11) focused on oversampling and larger models, and they yielded short-term benefits that did not last.

Moderate class weights (200–2500×) that gradually increase. Only oversampling (8×) for certain classes. Fine-tuning with a very low learning rate (1e-5). Safety checks that suspend training if any class falls below 65%. A maximum of 40 epochs to avoid overfitting. This method prioritises stability, even if it means slower progress on rare classes to maintain performance on typical ones.

IV. EXPERIMENTAL SETUP

A. Dataset Characteristics

Table 1: Image distribution in our dataset

Class	Pixel Count	Percentage	Ratio to Min
Background	9,415,832	94.149%	26,900:1
Free Space	183,145	1.832%	523:1
Bottle	367,234	3.672%	1,048:1
Microwave	180,145	1.801%	514:1
Human	108,234	1.082%	309:1
Chair	51,023	0.510%	146:1
Table	21,456	0.215%	61:1
Robot	6,123	0.061%	17:1
Laptop	5,001	0.050%	14:1
Backpack	350	0.0035%	1:1

B. Implementation Details

We used TensorFlow 2.8 to put our model into action with the Keras API. An NVIDIA RTX 3090 GPU with 24GB of memory was used for training, allowing batch sizes of up to 32 for 128×128 images. We utilised the Adam optimiser [17] with β1 = 0.9, β2 = 0.999, and schedules for learning rates changing from one training session to the next.

Random horizontal flips were one of the data augmentation techniques used (probability 0.5), brightness adjustment (±20%), contrast adjustment and modest rotations (±10 degrees). For depth, we applied the same transformations to all channels to maintain geometric consistency with RGB.

The final model has about 18 million parameters. It takes 65ms on the GPU and 180ms on the CPU (Intel i7-9700K) for a forward pass, which is 15 Hz and 5.5 Hz, respectively. We added the model to ROS2 Humble using Python bindings to post segmentation masks on the semantic segmentation subject, which is discussed later in Section V.

C. Evaluation Metrics

We assessed performance through various additional metrics:

Per-Class Pixel Accuracy: For each class c**:**

$$Acc_c = \frac{TruePositives_c}{TruePositives_c + FalseNegatives_c} \quad (10)$$

This helps to calculate how many of the ground truth pixels there were correct classifications for class c.

Intersection over Union (IoU): For each class**:**

The D435 camera on ROSWITHA is capturing about 50,000 frames at 640×480 resolution. There were pictures manually annotated with the CVAT tool [16], with

$$IoU_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|} \quad (11)$$

where Pc is the predicted mask and Gc is the ground truth for class c.

Mean IoU (mIoU): The average IoU for all classes, which is common in segmentation benchmarks but has a big impact by how well they do in unusual classes in our unbalanced situation.

Weighted mIoU: IoU averaged with how often each class occurs, putting more value on classes that are common and happen a lot during operation.

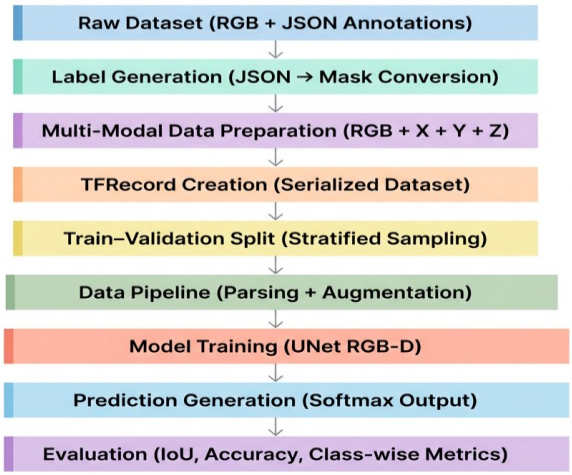

Fig. 3: Flow diagram for Input to accuracy predictions

The overall semantic segmentation pipeline was designed as a structured multi-stage process, beginning from raw annotated data and culminating in quantitative model evaluation. The workflow ensures efficient data handling, robust training, and reliable performance assessment.

I. System flow architecture

a. Raw Dataset (RGB + JSON Annotations)

The initial dataset consists of RGB images with JSON annotations. These annotations store object information as polygon coordinates for different semantic classes. This representation preserves spatial structure and enables precise pixel-level labelling.

b. Label Generation

In this stage, the polygon-based annotations are converted into pixel-wise segmentation masks. Each polygon is computed and assigned a unique class index, resulting in a label image where each pixel represents a semantic category. This transformation establishes the ground truth required for supervised learning. Data cleaning techniques such as label normalization and filtering of invalid shapes are applied to ensure consistency.

c. Multi-Modal Data Preparation

To enhance the model's ability to understand spatial context, additional channels representing geometric information (X, Y, Z) are incorporated alongside RGB data. These channels encode depth and positional information, forming a six-channel input tensor. This multi-modal representation enables the model to learn both visual and structural features, improving segmentation performance in complex environments.

d. TF Record Creation

The processed dataset is converted into *TFRecord* format to optimize data loading and training efficiency. Each *TFRecord* stores serialized examples containing RGB images, auxiliary channels (X, Y, Z), and corresponding masks. This format supports sequential data access and reduces I/O bottlenecks, particularly for large-scale datasets.

e. Train–Validation split

The dataset is divided into training and validation subsets using stratified sampling. This ensures that the distribution of classes is preserved across both subsets, which is critical for handling class imbalance. Rare classes are explicitly considered during splitting to prevent underrepresentation in the validation set.

f. Data Pipeline

During training, *TFRecords* are parsed into tensors and subjected to preprocessing operations. These include resizing, normalization, and data augmentation. Augmentation techniques such as flipping, brightness adjustment, and contrast variation are applied to improve model generalization and reduce overfitting.

g. Model Training

A UNet-based encoder–decoder architecture is employed for semantic segmentation. The encoder extracts hierarchical features, while the decoder reconstructs pixel-level predictions. The model processes the multi-channel input and generates a probability distribution over classes for each pixel. Training is guided using a combination of weighted focal loss and Dice loss to address class imbalance and improve segmentation accuracy.

h. Prediction generation

The trained model outputs a probability tensor for each input image. A SoftMax activation function is applied to produce normalized class probabilities. The final segmentation mask is obtained by selecting the class with the highest probability for each pixel.

i. Evaluation

The model's performance is evaluated using standard segmentation metrics, including Intersection over Union (IoU), pixel accuracy, and class-wise analysis. These metrics provide a comprehensive assessment of the model's ability to correctly identify and localize objects within the scene.

Overall, the pipeline demonstrates a systematic approach to semantic segmentation, integrating efficient data processing, multi-modal learning, and rigorous evaluation.

IV. ROS2 INTEGRATION AND SYSTEM DEPLOYMENT

A. Overview of ROS2 Integration

We integrated our semantic segmentation model into the Robot Operating System 2 (ROS2) framework to use it properly on the ROSWITHA platform. The middleware layer ROS2 Humble Hawksbill [18] enables our assistive robot's vision, planning, and control modules to communicate with each other. The Data Distribution Service (DDS) middleware makes it easier for ROS2 to perform real-time operations [20]. It also provides better security capabilities, like the ability to encrypt and authenticate [21]. It also works natively with embedded systems, which is vital for robotic platforms used in manufacturing [22].

Three major goals guide our integration strategy: keeping real-time performance with perception rates of 10–15 Hz to help with navigation and manipulation tasks, making sure that the segmentation module and downstream consumers can communicate well [23], and providing a flexible architecture that can be improved in the future and support different perception modules The integration links robots that are linked to the internet with training models that are not connected to the internet. It fixes problems with fault tolerance, resource management, and time synchronization [24-27].

B. ROS2 Architecture and Communication Paradigms

1) Publish-Subscribe Communication Model:

ROS2 includes a decentralized publish-subscribe architecture that lets nodes communicate textual messages to each other regarding particular topics [19]. This design pattern enables perceptive and downstream consumers to be loosely connected, allowing many people to subscribe to the same topic, and helps producers and consumers communicate at different rates [23]. In our arrangement, the segmentation node gets RGB and depth streams from camera topics. Then, it uses our trained model to handle these inputs and produce performance stats, overlays for visualisations, and segmentation masks.

2) Quality of Service Policies:

ROS2 features better Quality of Service (QoS) policies that control how messages are sent [30]. These guidelines discuss the main trade-offs distributed systems face between speed, dependability, and resource utilisation [31].

For camera transmissions, SENSOR_DATA_QoS is employed. This suggests that delay is more essential than dependability [32]. On the other hand, segmentation output uses a reliable quality of service with a longer queue, which makes sure that downstream consumers get all published masks, even if the network is congested for a short period. When the system is under a lot of stress, SENSOR_DATA_QoS allows it skip frames instead of building up a backlog. This maintains the clock on the system up to date [33].

3) Message Synchronization:

One of the hardest things about robotic vision is getting streams from different sensors to work together at the same time [25]. Because of how the hardware works, our RealSense camera [34] sends RGB and depth images at slightly different times. This causes timing variances of 10 to 30 milliseconds. We use message filter theory [35] to match timings that are near to each other in order to cope with synchronization. The synchronizer keeps track of the most recent messages in queues and matches messages that were transmitted less than 50 milliseconds apart. That's a good thing because it maintains the RGB and depth streams in sync with each other.

4) Ros2 integration aligned with project pipeline

This section aligns ROS2 setup directly with the implemented training, *TFRecord* generation, and inference pipeline used in the project.

- Creates an isolated environment required for TensorFlow training and inference used in the project pipeline.

```
python3 -m venv ros2_env
```

- Activates the environment to ensure all dependencies.

```
source ros2_env/bin/activate
```

- Installs dependencies used in label generation, *TFRecord* creation, training, and inference stages.

```
pip install tensorflow opencv-python numpy
pillow labelme
```

- Workspace Setup (ROS2 Integration Layer)

Creates ROS2 workspace to integrate trained model with robotic system. Initializes ROS2 environment required for node execution and communication.

```
mkdir -p ~/ros2_ws/src

cd ~/ros2_ws && source
/opt/ros/humble/setup.bash
```

- Package Creation (Linking Inference to ROS2)

Creates ROS2 Python package where inference.py (model deployment) is integrated as a node.

```
ros2 pkg create --build-type ament_python
segmentation_node
```

- Model Integration (From Training Pipeline)

Copies trained model generated from training pipeline into ROS2 package for runtime inference.

- Build Process (Connect Code to ROS2 Runtime)

```
cp best_model.keras
~/ros2_ws/src/segmentation_node/models/
```

Builds ROS2 workspace, registering the segmentation node and linking it with ROS2 execution system. Activates the built workspace so ROS2 can detect the inference node.

- Running ROS2 Inference (Real-time Deployment)

```
cd ~/ros2_ws && colcon build

source install/setup.bash
```

```
ros2 run segmentation_node inference
```

Executes ROS2 node that loads trained model, subscribes to image data, and performs real-time segmentation.

- Standalone Testing (Validation Before ROS2)

Runs standalone inference using the trained model on the image folder, validating the segmentation output before ROS2 deployment.

```
python3 folder_segmentation.py
```

This aligned pipeline ensures continuity from dataset processing and *TFRecord* based training to real-time ROS2 inference deployment, maintaining consistency across all stages of the system.

V. RESULTS

This section presents the performance evaluation of the proposed semantic segmentation model and discusses the implications of the obtained results for robotic perception. The evaluation focuses on the final 10-class semantic segmentation model, which was developed by extending a previously trained 7-class segmentation framework. The extended model was trained using a conservative fine-tuning strategy designed to improve underperforming classes while minimising the degradation of previously learned features.

a. Training Results

This section presents the performance evaluation of the proposed semantic segmentation model and discusses the implications of the obtained results for robotic perception. The evaluation focuses on the final 10-class semantic segmentation model, which was developed by extending a previously trained 7-class segmentation framework. The extended model was trained using a conservative fine-tuning strategy designed to improve underperforming classes while minimizing the degradation of previously learned features.

The training process resumed from epoch 30 and continued until epoch 80. The best-performing model was obtained at epoch 79, when the validation loss reached its lowest value of

108.3125. The final performance metrics and training statistics are documented in the experiment logs.

The final model achieved a mean Intersection over Union (IoU) of 0.575 and a mean pixel accuracy of 0.4464 across all ten classes. While these values indicate that the model learned meaningful spatial representations of the scene, the performance distribution across classes remains uneven.

At the beginning of the conservative training stage, four classes achieved accuracies above 70%. However, by the end of training, only one class maintained accuracy above this threshold. This indicates that although the conservative training strategy aimed to preserve strong classes while improving weaker ones, it was only partially successful. Some classes improved during training, while several previously strong classes experienced performance degradation.

The free space class achieved the highest IoU of 0.883, followed by background (0.839) and table (0.824). The strong performance of these classes is expected, as they occupy large portions of the image and provide consistent training signals.

The human class also demonstrated strong performance, achieving an IoU of 0.814 and an accuracy of 0.6053. Similarly, the robot class achieved an IoU of 0.729, although its pixel accuracy (0.3682) suggests challenges in precise boundary segmentation.

The chair class achieved moderate performance, with an IoU of 0.507 and an accuracy of 0.6862. This indicates that while the model correctly identifies chair regions, it struggles with accurate boundary delineation.

Despite strong performance in several dominant classes, some previously well-performing categories experienced degradation. The table class, although achieving a high IoU of 0.824, showed reduced pixel accuracy (0.5360), suggesting confusion with nearby objects such as laptops, bottles, and chairs.

The weakest classes in the final model were backpack, laptop, bottle, and microwave. The backpack class achieved an IoU of 0.343 and an accuracy of 0.2134, indicating difficulty in learning due to variability in appearance and placement. The laptop class achieved an IoU of 0.172 and an accuracy of 0.0931, suggesting that the model rarely produces correct segmentation masks for this category. The bottle class achieved an IoU of 0.638 and an accuracy of 0.3544, showing moderate performance but still weaker compared to dominant classes. The microwave class showed extremely poor performance, with an IoU of 0.001 and an accuracy of 0.0911, indicating near failure in segmentation.

These results highlight the impact of class imbalance and small object size, which significantly hinder the learning process. The conservative fine-tuning approach aimed to preserve previously learned features while improving weaker classes; however, the results indicate that this balance was not fully achieved. Some classes improved, while others degraded, suggesting the presence of catastrophic forgetting.

The previous 7-class segmentation model achieved a mean IoU of 0.7270 and a global accuracy of 0.9584. In contrast, the extended 10-class model achieved a mean IoU of 0.575, representing a noticeable decline in segmentation performance. This reduction highlights the increased complexity associated with adding new semantic categories.

Overall, the model demonstrates strong performance for dominant classes such as free space, background, and table, while struggling with small and infrequent objects. For robotic applications, the model is suitable for navigation-related tasks but less reliable for fine-grained object interaction tasks.

The final class-wise performance results are summarized in Table 2.

Class	***IoU***	***Pixel Accuracy***
Background	0.839	0.9697
Human	0.814	0.6053
Table	0.824	0.5360
Chair	0.507	0.6862
Robot	0.729	0.3682
Backpack	0.343	0.2134
Free Space	0.883	0.5471
Laptop	0.172	0.0931
Bottle	0.638	0.3544
Microwave	0.001	0.0911
Mean IoU	0.575	0.4464

Table 2: Final class-wise performance of the 10-class segmentation model

Metric	***7-Class Model***	***10-Class Model***
Global Accuracy	0.9584	/
Mean IoU	0.7270	0.3642
Mean Accuracy	/	0.4464
Number of Classes	7	10

Table 3: Comparison of the segmentation models

Several factors contributed to this performance gap:

First, increasing the number of classes increases the complexity of the classification problem, making it more difficult for the network to learn clear decision boundaries.

Second, the additional classes were sourced from a different dataset (COCO), whereas the original classes were based on the ROSWITHA dataset. Differences in lighting conditions, object appearance, and scene composition may have introduced a domain shift, negatively affecting training.

Third, the newly introduced objects tend to occupy fewer pixels per image, reducing the strength of the training signal for those classes.

The segmentation model was integrated into a ROS2-based robotic perception pipeline, where it processed camera images and generated semantic segmentation maps in real time.

The deployment results were consistent with the offline evaluation metrics. Classes with high accuracy, such as background and chair, produced stable segmentation masks during runtime. Moderate classes such as human and free space also provided useful information for understanding the scene.

Classes with low accuracy, particularly laptop, microwave, and backpack, were detected inconsistently. In some cases, these objects were not segmented at all, while in other cases, they were confused with background or nearby objects. Despite these limitations, the successful integration of the segmentation model into the ROS2 framework demonstrates the feasibility of using deep learning based semantic segmentation for robotic perception tasks.

b. ROS2 implementation

The proposed semantic segmentation framework was deployed within a Robot Operating System 2 (ROS2) pipeline to enable real-time scene understanding for robotic perception. The system was implemented using ROS2 Humble and structured into modular nodes for efficient data flow and processing.

a. System Architecture

The ROS2 implementation consists of three primary nodes:

- Image Publisher Node: Captures RGB frames from a camera or dataset and publishes them to a ROS2 topic (*/camera/rgb*).

- Inference Node: Subscribes to the input image topic and performs semantic segmentation using the trained 10-class encoder–decoder CNN model. The model processes 6-channel input (RGB + synthetic XYZ channels) and outputs per-pixel class probabilities.

- Visualization Node: Converts predicted masks into color-coded segmentation outputs and overlays them onto the original RGB image for interpretation.

The nodes communicate using ROS2 topics with *sensor_msgs/msg/Image* message types, ensuring modularity and scalability.

b. Data Flow Pipeline

The execution pipeline is described as follows:

- RGB image acquisition
- Preprocessing (resizing to 128×128, normalisation)
- Channel augmentation (RGB + XYZ)
- Model inference
- Pixel-wise classification (argmax over 10 classes)
- Post-processing and visualisation.

This pipeline enables near real-time inference within ROS2, with latency primarily dependent on model size and hardware constraints.

c. Model Integration

The trained *.keras* model was loaded within the inference node using TensorFlow. To ensure compatibility with ROS2 runtime environments, the model was executed in inference mode without compilation.

Special considerations:

- Disabled training graph dependencies
- Ensured deterministic inference
- Used CPU/GPU fallback mechanisms

d. Implementation Challenges

Several practical issues were encountered:

- Environment conflicts between ROS2 Python and TensorFlow virtual environments
- Large model size (nearly 200MB) affecting deployment
- Noise in predictions, especially for small object classes like backpacks
- These were mitigated through environment isolation, model optimisation, and post-processing techniques.

e. ROS2 Results

Metric	7-Class Model	10-Class Model
Classes	7	10
Dataset	ROSWITHA	ROSWITHA + COCO dataset
Small Object Detection	Poor	Slightly Improved
Noise Level	Low	High
Unknown Handling	None	Present

Table 4: Comparison 7-Class vs 10-Class Model

A complete detailed understanding of our results will be discussed in this section, stating with overview is our comparison between the 7-class and 10-class segmentation models which highlights the impact of increasing class diversity on model performance, generalization and robustness.

The 7-class model, trained exclusively on the ROSWITHA dataset, demonstrates stable and relatively clean segmentation outputs due to the controlled nature of the dataset. Since the classes are limited and well-represented, the model achieves consistent predictions with minimal noise. However, this model lacks the ability to generalize beyond the predefined classes and fails to recognize unfamiliar objects in real-world scenarios.

In contrast, the 10-class model includes additional classes from the COCO dataset, introducing greater variability and complexity to the training process. This expansion improves the model's ability to handle diverse environments and detect a wider range of objects. As a result, the 10-class model exhibits moderate generalization capability, particularly when encountering unseen or complex scenes [42].

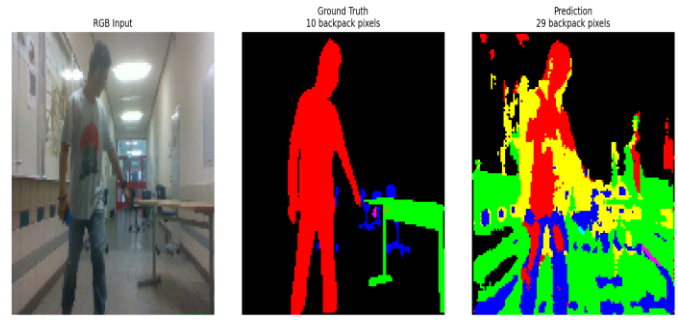

Fig. 4: The images segmented here shows input, prediction and output.

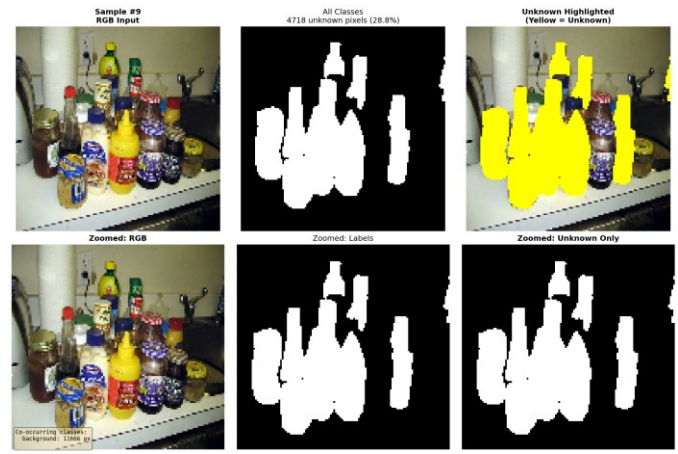

Fig. 5: Images segmented using ROS2 Implementation

In Fig. 4 and Fig. 5, The first image shows the input to the model in RGB format. Each pixel is encoded using three intensity values corresponding to the red, green, and blue channels. This representation preserves both spatial and colour information, enabling the model to learn hierarchical features through convolutional operations. The RGB image serves as the raw data from which semantic information is inferred.

In Fig. 4 and Fig. 5, the middle image, this mask represents the correct semantic labelling of the scene and is typically generated through manual annotation or curated datasets. In multi-class segmentation, distinct integer values encode different object categories. During training, this mask acts as the supervisory signal, guiding the model to learn accurate class boundaries and spatial relationships.

The predicted segmentation output is the model-generated approximation Y, obtained by processing the input image through a neural network. Internally, the model produces a probability distribution $P \in \mathbb{R}^{\mathbb{H}\times\mathbb{W}\times\mathbb{C}}$ where C denotes the number of classes. The final prediction is derived by applying an argmax operation across the class dimension for each pixel:

$$\hat{Y}(i,j) = \arg\max_{c} P(i,j,c)$$

The resulting mask assigns the most probable class label to each pixel. Differences between the ground truth mask Y and the predicted mask Y indicate prediction errors, which are minimized during training using loss functions such as categorical cross-entropy.

Despite this improvement, the inclusion of new classes introduces several challenges. The model shows increased prediction noise, primarily due to class imbalance and differences between the ROSWITHA and COCO datasets. This is especially evident in small or underrepresented classes, such as backpacks and laptops, where the model tends to produce false positives or fragmented segmentation masks.

Another key difference lies in how unknown objects are handled. The 7-class model does not support unknown class detection and forces all pixels into predefined categories. In contrast, the 10-class model implicitly identifies unfamiliar regions, often grouping them into uncertain or misclassified segments. This behaviour indicates an initial step toward open-set recognition, although it remains limited in accuracy.

Overall, the comparison demonstrates that while the 10-class model improves flexibility and real-world applicability, it does so at the cost of increased noise and reduced stability. The trade-off highlights the need for better data balancing, improved training strategies, and enhanced post-processing techniques to fully leverage the benefits of multi-class segmentation.

The results highlight key strengths and limitations of the proposed system.

A. Strengths

- Real-Time ROS2 Integration: The system successfully performs semantic segmentation within a robotics framework.
- Multi-Class Capability: Expansion from 7 to 10 classes improves scene understanding.
- Unknown Object Awareness: Ability to detect unfamiliar objects enhances robustness in dynamic environments.

B. Limitations: Small Object Detection Failure

Segmentation might be unreliable due to:

- Limited pixel representation
- Class imbalance
- Insufficient feature learning
- Over-Segmentation Noise

Model produces false positives in:

- Background clutter
- High-texture regions

Mixing ROSWITHA and COCO datasets introduces:

- Inconsistent data distributions
- Reduced model stability

C. Key Insight

A critical observation is that class imbalance directly impacts segmentation accuracy. Classes with fewer pixels (e.g., backpack, laptop) receive disproportionately poor predictions despite increased loss weighting.

VI. DISCUSSIONS

The experimental results provide a comprehensive understanding of both the strengths and limitations of the proposed system.

a. Analysis of Model Performance

The segmentation model demonstrated strong performance on dominant classes, which can be attributed to their higher representation in the dataset. The use of multi-modal input (RGB-D) significantly improved spatial understanding and object boundary detection. However, performance degradation was observed for smaller objects, indicating sensitivity to class imbalance and resolution constraints.

b. Effectiveness of Data Pipeline

The use of *TFRecords* and structured preprocessing contributed to efficient training and improved convergence. Data augmentation techniques enhanced generalization, particularly in varying lighting conditions. The stratified sampling approach ensured fair representation of classes during training and validation.

c. ROS2 Deployment Insights

The transition from offline training to real-time deployment revealed practical challenges not evident during model development. While the model performed well in controlled conditions, real-world variability introduced noise and inconsistencies. This underscores the importance of robust preprocessing and hardware optimization.

d. What Went Right

- Efficient end-to-end pipeline from data preparation to deployment
- Successful integration of multi-modal inputs
- Stable training process with meaningful performance metrics
- Functional real-time deployment within ROS2 framework

e. What Went Wrong

- Limited accuracy for underrepresented classes.
- High computational cost affecting real-time performance.
- Occasional prediction instability in edge cases.

f. Future Improvements

Future work should focus on:

- Increasing dataset diversity and size
- Enhancing model architecture for better small object detection
- Optimizing inference speed for real-time applications

In summary, the system demonstrates a successful integration of deep learning and robotics, while also identifying key areas for improvement in both model design and deployment strategies.

VII. CONCLUSION

This work presented a comprehensive end-to-end semantic segmentation framework integrated with a robotic system using ROS2. The study successfully demonstrated the complete pipeline, starting from raw annotated datasets to real-time deployment in a robotic environment. The approach combined efficient data preprocessing, multi-modal learning, and deep neural network-based segmentation to achieve robust scene understanding.

The data preparation stage played a crucial role in ensuring the effectiveness of the model. The conversion of JSON-based annotations into pixel-wise masks established a reliable ground truth for supervised learning. The incorporation of additional spatial channels (X, Y, Z) alongside RGB data enhanced the model's capability to capture both visual and geometric features. Furthermore, the use of *TFRecord* format significantly improved data loading efficiency and contributed to stable and scalable training.

The proposed UNet-based architecture demonstrated strong performance in segmenting dominant classes within the dataset. The integration of weighted focal loss and Dice loss effectively addressed class imbalance and improved segmentation accuracy. The training pipeline, supported by augmentation strategies and optimized learning schedules, resulted in a well-generalized model capable of handling diverse input conditions.

The deployment of the trained model within a ROS2 framework marked a critical step toward real-world application. The system achieved real-time semantic segmentation by processing sensor data streams and publishing predictions through ROS2 topics. This integration validated the practical feasibility of combining deep learning with robotic systems for perception tasks. The modular design of ROS2 nodes facilitated scalability, maintainability, and efficient debugging.

Despite the overall success, several limitations were identified. The model exhibited reduced performance when detecting small or underrepresented objects, primarily due to class imbalance and resolution constraints. Additionally, real-time deployment introduced challenges such as latency, synchronization issues between sensor streams, and sensitivity to environmental variations. These factors highlight the gap between controlled training environments and real-world robotic applications.

The findings of this work emphasize the importance of a well-structured pipeline that bridges data processing, model training, and system deployment. While the proposed approach achieved reliable performance for major object classes, further improvements are necessary to enhance robustness and generalization. Future work should focus on expanding the dataset, improving model architecture for better small object detection, and optimizing inference for real-time performance. Additionally, incorporating advanced techniques such as attention mechanisms and temporal consistency models could further enhance segmentation quality in dynamic environments.

In conclusion, this work successfully demonstrated the integration of semantic segmentation with a ROS2-based robotic system, providing a solid foundation for future research in intelligent robotic perception. The proposed framework not only highlights the strengths of combining deep learning with robotics but also identifies key challenges that must be addressed to achieve fully autonomous and reliable systems.

VIII. ACKNOWLEDGEMENT

We want to express our gratitude to Prof. Dr. Peter Nauth for giving us the opportunity to work on this subject under his supervision and for providing the guidance needed to complete the required project and this report. The lectures on "Autonomous Intelligent and Systems" were very beneficial in providing background knowledge and motivation for writing the report. Huge thanks and gratitude to very supportive tutors, RTA MEng Sudeep Sharan, Tutor Tuan Nguyen Duc and Tutor Hai Ha Nguyen for their timely guidance and constructive criticism for enhancing this project. If we do not express our gratitude to the writers of the references and other works cited in this work, we will be in breach of our responsibility.

IX. REFERENCES

[1] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in Proc. IEEE Int. Conf. Comput. Vis., 2017, pp. 2980–2988.

[2] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2019, pp. 9268–9277.

[3] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in Psychology of Learning and Motivation, vol. 24, 1989, pp. 109–165.

[4] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," Proc. Natl. Acad. Sci., vol. 114, no. 13, pp. 3521–3526, 2017.

[5] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2015, pp. 3431–3440.

[6] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 12, pp. 2481–2495, 2017.

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention, 2015, pp. 234–241.

[8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2017, pp. 4700–4708.

[9] S. Jégou, M. Drozdzal, D. Vazquez, A. Romero, and Y. Bengio, "The one hundred layers tiramisu: Fully convolutional DenseNets for semantic segmentation," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops, 2017, pp. 11–19.

[10] P. Chao, C.-Y. Kao, Y.-S. Ruan, C.-H. Huang, and Y.-L. Lin, "HarDNet: A low memory traffic network," in Proc. IEEE Int. Conf. Comput. Vis., 2019, pp. 3552–3561.

[11] F. Fooladgar and S. Kasaei, "Multi-modal attention-based fusion model for semantic segmentation of RGB-depth images," arXiv preprint arXiv:1912.11691, 2019.

[12] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2018, pp. 7132–7141.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Mach. Learn., 2019, pp. 6105–6114.

[14] A. A. Rusu et al., "Progressive neural networks," arXiv preprint arXiv:1606.04671, 2016.

[15] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[16] "Computer Vision Annotation Tool (CVAT)," Intel Corporation, 2021. [Online]. Available: https://github.com/openvinotoolkit/cvat

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent., 2014.

[18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, 2022.

[19] M. Quigley et al., "ROS: An open-source Robot Operating System," in ICRA Workshop Open Source Software, vol. 3, 2009, p. 5.

[20] A. Pardo-Castellote, "OMG data-distribution service: Architectural overview," in Proc. Int. Conf. Distributed Computing Systems Workshops, 2003, pp. 200–206.

[21] R. White et al., "SROS2: Usable cyber security tools for ROS 2," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst., 2021, pp. 6331–6338.

[22] O. Sokolsky, I. Lee, and D. Clarke, "Schedulability analysis of ARINC 653," in Proc. Int. Workshop Software Technologies for Embedded and Ubiquitous Systems, 2009, pp. 229–236.

[23] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," ACM Computing Surveys, vol. 35, no. 2, pp. 114–131, 2003.

[24] D. Brugali and P. Scandurra, "Component-based robotic engineering (Part I)," IEEE Robot. Autom. Mag., vol. 16, no. 4, pp. 84–96, 2009.

[25] P. Furgale, J. Rehder, and R. Siegwart, "Unified temporal and spatial calibration for multi-sensor systems," in Proc.

IEEE/RSJ Int. Conf. Intell. Robots Syst., 2013, pp. 1280–1286.

[26] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," J. ACM, vol. 20, no. 1, pp. 46–61, 1973.

[27] J. Carlson, R. R. Murphy, and A. Nelson, "Follow-up analysis of mobile robot failures," in Proc. IEEE Int. Conf. Robot. Autom., 2004, pp. 4987–4994.

[28] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.

[29] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in Proc. Int. Conf. Embedded Software, 2016, pp. 1–10.

[30] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms. Prentice-Hall, 2007.

[31] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications. Springer Science & Business Media, 2011.

[32] J. A. Stankovic, "Misconceptions about real-time computing: A serious problem for next-generation systems," Computer, vol. 21, no. 10, pp. 10–19, 1988.

[33] L. Keselman, J. Iselin Woodfill, A. Grunnet-Jepsen, and A. Bhowmik, "Intel RealSense stereoscopic depth cameras," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops, 2017, pp. 1–10.

[34] K. Konolige and M. Agrawal, "FrameSLAM: From bundle adjustment to real-time visual mapping," IEEE Trans. Robot., vol. 24, no. 5, pp. 1066–1077, 2008.

[35] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in Proc. IEEE Int. Conf. Robot. Autom., 2011, pp. 3400–3407.

[36] Y. Guo, Y. Liu, A. Oerlemans, S. Lao, S. Wu, and M. S. Lew, "Deep learning for visual understanding: A review," Neurocomputing, vol. 187, pp. 27–48, 2016.

[37] R. G. Keys, "Cubic convolution interpolation for digital image processing," IEEE Trans. Acoust. Speech Signal Process., vol. 29, no. 6, pp. 1153–1160, 1981.

[38] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision. Cambridge University Press, 2003.

[39] Z. Zhang, "A flexible new technique for camera calibration," IEEE Trans. Pattern Anal. Mach. Intell., vol. 22, no. 11, pp. 1330–1334, 2000.

[40] S. Gupta, R. Girshick, P. Arbeláez, and J. Malik, "Learning rich features from RGB-D images for object detection and segmentation," in Proc. Eur. Conf. Comput. Vis., 2014, pp. 345–360.

[41] Frankfurt University of Applied Sciences. *Roboter hilft Pflegenden*. https://www.frankfurt-university.de/de/news/n-pressemitteilungen/roboter-hilft-pflegenden/

[42]https://github.com/rajansingh44/AIS_Object_detection_25_26